Cloud Messenger™

Learn | Build | Deploy | Grow

EDITION 2026

NGINX

PRODUCTION HANDBOOK

Web Server • Reverse Proxy • Load Balancing
Troubleshooting • Zero-Downtime • Docker & Kubernetes

10 Visual Pages | Real Commands | 502/504 Playbooks | AWS + Docker + K8s



#buildwithCloud-Messenger

cloudmessenger.in

Learn • Build • Visualize • Troubleshoot • Grow

Cloud Messenger™ — Practical. Production. Professional.

INSIDE THE HANDBOOK

NGINX Production Handbook • Cloud Messenger™

Ten illustrated pages that take you from first install to production-grade operations.
Tap any chapter below to jump straight to its page.

1 **NGINX in Production** — p. 3
What NGINX does in modern infrastructure: request flow, roles and architecture

2 **Key Advantages** — p. 4
Concurrency, low memory, reverse proxy, load balancing, caching, graceful reloads

3 **Linux Setup Guide** — p. 5
19 hands-on steps to install and run an NGINX server on Linux

4 **Web Server & Production Notes** — p. 6
Commands, files, ports, error codes, interview questions, 502 case

5 **Nginx-Verse: 5 Deployment Paths** — p. 7
GitHub Pages, Docker, ECS + ECR, Jenkins CI/CD and EC2

6 **Production Load Balancing** — p. 8
Upstreams, algorithms, weights, backup servers, failure handling

7 **502 & 504 Troubleshooting** — p. 9
Two incident scenarios with 10-step investigation paths

8 **NGINX as a Reverse Proxy** — p. 10
proxy_pass, headers, buffering, timeouts, upstream debugging

9 **Zero-Downtime Deployments** — p. 11
Reloads, rolling, blue-green, canary, draining, rollback

10 **NGINX with Docker & Kubernetes** — p. 12
Containers, volumes, networking, Ingress, TLS, routing rules

REMEMBER

Zero-downtime is a process, not a single command. Test, deploy gradually, monitor, and roll back fast if needed.

NGINX IN PRODUCTION



WHAT NGINX DOES IN MODERN INFRASTRUCTURE

WEB SERVER
Serves websites and web applications.

REVERSE PROXY
Forwards client requests to backend servers securely.

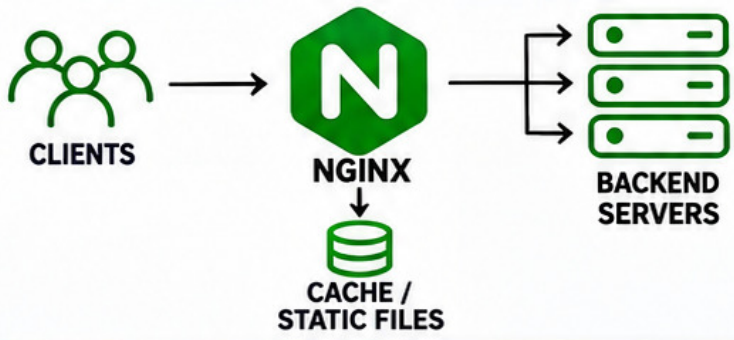
LOAD BALANCER
Distributes traffic across multiple backend servers.

API GATEWAY USE CASES
Handles routing, rate limiting, authentication, and more.

TLS TERMINATION
Offloads SSL/TLS encryption and improves performance.

STATIC CONTENT DELIVERY
Delivers static files fast and efficiently with low resources.

TYPICAL PRODUCTION REQUEST FLOW



NGINX VERSUS APPLICATION SERVERS

NGINX (EDGE / INFRASTRUCTURE)	APPLICATION SERVERS (BUSINESS LOGIC)
✓ HIGH PERFORMANCE	✓ PROCESS BUSINESS LOGIC
✓ LOW MEMORY USAGE	✓ GENERATE DYNAMIC RESPONSES
✓ HANDLES THOUSANDS OF CONNECTIONS	✓ DATABASE OPERATIONS
✓ EXCELLENT FOR STATIC CONTENT	✓ USER SESSIONS
✓ SSL OFFLOADING	✓ APPLICATION-SPECIFIC FUNCTIONS
✓ LOAD BALANCING	✓ MORE RESOURCE INTENSIVE
✓ REVERSE PROXY	✓ NOT IDEAL AS EDGE PROXY
✓ LOWER INFRASTRUCTURE COSTS	

PRODUCTION ARCHITECTURE OVERVIEW



Cloud Messenger™
Learn | Build | Deploy | Grow

NGINX KEY ADVANTAGES

A Powerful Web Server and Reverse Proxy

that keeps your websites fast, secure and reliable.



1 High Concurrency Handling



- Thousands of users at the same time.
- Handles many connections without creating a new thread for each request.

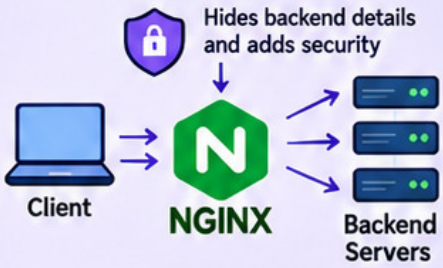
Handles tens of thousands of simultaneous connections using an event-driven, non-blocking model.

2 Low Memory Footprint



Requires minimal CPU and RAM resources compared to traditional servers like Apache.

3 Built-In Reverse Proxy



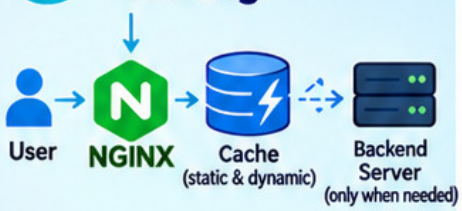
Acts as a secure intermediary that hides backend server details and passes client requests efficiently.

4 Traffic Load Balancing



Distributes incoming network traffic across multiple backend servers using methods like Round Robin or Least Connections to prevent overloads.

5 Fast Content Caching



Stores frequently requested static files and dynamic data locally to serve subsequent users instantly without hitting backend application servers.

6 Graceful Reloads



Updates configuration files and applies changes on the fly with zero downtime for active connections.

NGINX – More Performance. Less Resource Usage. Greater Control.

Cloud Messenger™
Learn | Build | Deploy | Grow

Let's Create an NGINX SERVER on Linux!

Build a Super Fast Website with NGINX





What is NGINX? | NGINX is a high-performance web server, reverse proxy, load balancer and HTTP cache. Many big websites use it!

Step-by-Step: Install & Run NGINX on Linux

1 Become Root User
Get super powers!



```
$ sudo su
```

2 Update Packages
Keep your system updated!



```
$ sudo yum update -y
```

3 Install NGINX
Let's install the web server!

NGINX

```
$ sudo yum install nginx -y
```

4 Go to usr Directory
Go to /usr directory!

/usr

```
$ cd usr
```

5 List Contents of usr
See what's inside /usr!



```
$ ls
```

6 Go to share Directory
Go inside /usr/share!

/usr/share

```
$ cd share
```

7 List Contents of share
See what's inside /usr/share!



```
$ ls
```

8 Go to nginx Directory
Go inside /usr/share/nginx!

nginx

```
$ cd nginx
```

9 List Contents of nginx
See what's inside /usr/share/nginx!



```
$ ls
```

10 html Directory
This is where web files live!

html

```
$ ls  (you will see: html)
```

11 Print Working Directory
Confirm current location!



```
$ pwd
```

12 Current Path
This is your web root path!

```
/usr/share/nginx/html
```

13 Create index.html File
Create your main page file!



```
$ touch index.html
```

14 Edit index.html File
Open file in vi editor!



```
$ vi index.html
```

15 Paste HTML Code
Paste your website code in file!

</>

```
<html>
  <body>
    <h1>Welcome to NGINX Server!</h1>
    <p>#BuildWithCloudMessenger 🚀</p>
  </body>
</html>
```

16 This will run as Main Website File
index.html is the default file!

```
/usr/share/nginx/html/index.html
```

17 Start NGINX Service
Start the web server!



```
$ systemctl start nginx
```

18 Enable NGINX on Boot
Start automatically on reboot!



```
$ systemctl enable nginx
```

19 Command History
See all commands you ran!



```
$ history
```

Test in Browser
Open this in your browser!

```
http://<YOUR-SERVER-IP>
```

You should see:
Welcome to NGINX Server!
#BuildWithCloudMessenger 🚀

Useful NGINX Commands

Restart	Stop	Reload	Status	Test Config
Reload everything	Stop the server	Apply changes	Check status	Test configuration
`$ systemctl restart nginx`	`$ systemctl stop nginx`	`$ systemctl reload nginx`	`$ systemctl status nginx`	`nginx -t`

Real-World Use Case



NGINX handles high traffic, balances load, serves static files fast, and makes your apps secure & scalable!

Why DevOps Teams Love NGINX?

- Super Fast Performance
- Handles High Traffic Easily
- Great for Load Balancing
- Works with Docker & K8s
- Secure with SSL/TLS

Pro Tip

Make sure your Security Group / Firewall allows HTTP (80) and HTTPS (443) traffic!

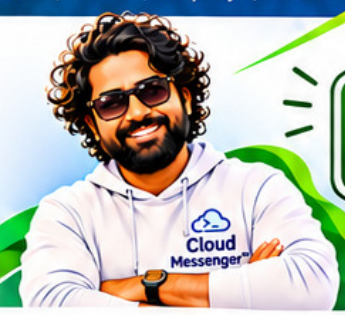


That's it! 🚀 Your NGINX website is live and running! ✅

#BuildWithCloudMessenger

NGINX
WEB SERVER & PRODUCTION NOTES

Complete Guide: Concepts • Commands • Use Cases • Troubleshooting



1 What is Nginx ?

- Nginx is a high performance web server used for:
  - Web Server
  - Reverse Proxy
  - Load Balancer
  - HTTP Cache

2 Real Company Use Cases

- **E-commerce** : Handle website traffic
- **Banking** : Secure requests
- **Streaming** : Fast content delivery
- **IT Companies** : Manage traffic

3 Key Features

- Fast & Lightweight
- Reverse Proxy
- Load Balancing
- SSL/TLS Security
- Caching
- Rate Limiting
- Compression
- High Availability

4 Architecture Example

Users → Nginx (Reverse Proxy + Load Balancer) → App Server (Node.js/etc) / Database (RDS)

5 Important Commands

- sudo systemctl start nginx
- sudo systemctl restart nginx
- sudo systemctl status nginx
- sudo nginx -t (Check Config)
- sudo systemctl reload nginx

6 Important Files & Ports

- Ports : 80 (HTTP), 443 (HTTPS)
- Config : /etc/nginx/nginx.conf
- Logs : /var/log/nginx/access.log
  /var/log/nginx/error.log

7 Common Error Codes

- 404 – Page Not Found
- 502 – Bad Gateway
- 503 – Service Unavailable
- 504 – Gateway Timeout

8 Production Troubleshooting Steps

1. Check Nginx service : systemctl status nginx
2. Check Logs : tail -f /var/log/nginx/error.log
3. Check Backend App
4. Check Ports : ss -tulnp
5. Fix & Verify

9 Interview Questions (1–15)

1. What is Nginx?
2. Why use Nginx over Apache?
3. What is Reverse Proxy?
4. Default Port?
5. What is Load Balancing?

... (6–15)

10 Real-Time Case : 502 Bad Gateway

Problem : Website shows 502 error.

Steps :
1. Check Nginx & Backend
2. Check Logs
3. Restart App
4. Reload Nginx
5. Verify Website

→ 502 Bad Gateway

Remember :

Nginx receives the request, forwards it to backend and returns response.

NGINX – At a Glance

High Performance | Reliable Reverse Proxy | Efficient Load Balancing | Built-in Security (SSL/TLS) | Caching & Compression | Production Ready

Nginx-Verse

5 Ways to Deploy Nginx

Same Nginx | Different Worlds | Real DevOps Skills



- LEARN
- PRACTICE
- IMPLEMENT
- DOCUMENT
- SHOWCASE



1 Nginx Server through GitHub

Host a Static Nginx Webpage using GitHub Pages

Nginx Static Files (HTML, CSS, etc.)
↓
GitHub Repository (nginx content)
↓
GitHub Pages (Static Website)
↓

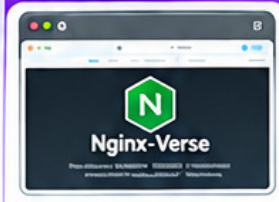
https://<username>.github.io/

NGINX
Welcome to Nginx-Verse
Deployed using GitHub Pages

Key Steps

- Create GitHub repository
- Add Nginx static files
- Enable GitHub Pages
- Configure index.html
- Publish and access the site

Output



https://divyapriya250413.github.io/nginx-server-visualization/

Nginx website accessible on GitHub Pages

2 Nginx Server through Docker

Run Nginx using Docker containerization

Dockerfile (from Nginx image)
↓
Docker Image (nginx)
↓
Docker Container (Running Nginx)
↓

http://localhost:8080

NGINX
Running in Docker

Key Steps

- Write Dockerfile
- Build Docker image
- Run Nginx container
- Map port (8080:80)
- Access Nginx in browser

Output

```
$ docker build -t nginx-app .
$ docker run -d -p 8080:80
  nginx-app
Nginx is running on
http://localhost:8080
```

NGINX
Running in Docker Container

Nginx running successfully in Docker container

3 Nginx Server through ECS & ECR

Build, Push and Deploy Nginx on AWS ECS using ECR

Docker Image (Nginx) → Amazon ECR (Store Image)
↓
Amazon ECS (Deploy Container)
↓
Application Load Balancer
↓
Nginx Task 1 | Nginx Task 2

Key Steps

- Build Docker image for Nginx
- Push image to Amazon ECR
- Create ECS cluster & task definition
- Deploy Nginx service
- Configure ALB and access the site

Output

Application Load Balancer — Active

nginx-alb-123456.ap-south-1.elb.amazonaws.com

NGINX
Running on AWS ECS (with ALB)

Nginx running on ECS with ECR and accessible via ALB

4 Nginx Server through Jenkins

Automate Nginx Build & Deploy using Jenkins (CI/CD)

GitHub (Source Code) → Jenkins (Build & Deploy) → Nginx (Deployed)

1. Pull Code from GitHub
2. Build Docker Image
3. Push to ECR / Deploy to Target
4. Deploy Nginx (EC2/ECS)

↓

jenkins CI/CD

NGINX
Deployed via Jenkins CI/CD

Key Steps

- Create Jenkins pipeline (Jenkinsfile)
- Pull code from GitHub
- Build Docker image / Deploy
- Automate deployment
- Check build and deployment logs

Output

Jenkins Pipeline

Build Successful

Checkout → Build → Deploy → Success

```
[INFO] Pulling code from GitHub...
[INFO] Building Docker image...
[INFO] Deploying Nginx...
[INFO] Nginx deployed successfully!
```

Nginx automatically deployed using Jenkins pipeline

5 Nginx Server through EC2 Server

Install and Configure Nginx on an AWS EC2 Instance

Amazon EC2 (Windows/Linux)
↓

```
sudo yum install nginx
(or)
sudo apt install nginx
sudo systemctl start nginx
sudo systemctl enable nginx
```

↓

http://<EC2-Public-IP>

NGINX
Running on EC2 Instance

Key Steps

- Launch EC2 instance (Amazon Linux)
- Connect using SSH
- Install Nginx
- Start and enable service
- Configure security group (port 80)
- Access Nginx in browser

Output

```
[ec2-user@ip-xxx ~]$ sudo
systemctl status nginx
● nginx.service - running
```

NGINX
Running on EC2 Instance

Nginx running on AWS EC2 and accessible via public IP / DNS

What You Learn?

- Version Control (Git & GitHub)
- Containerization (Docker)
- AWS Services (ECR, ECS, EC2, ALB)
- CI/CD Automation (Jenkins)
- Real World Deployment & Troubleshooting

#buildwithcloud-messenger

PRODUCTION LOAD BALANCING

USING NGINX – CONCEPTS, METHODS, CONFIG & BEST PRACTICES



WHAT IS LOAD BALANCING?

Distributes incoming traffic across multiple backend servers using Nginx.

KEY BENEFITS

- High Availability
- Better Performance
- Scalability
- Fault Tolerance

COMMON USE CASES

- E-commerce websites
- Banking & Financial apps
- Streaming platforms
- IT / Enterprise applications

KEY NGINX FEATURES

- Lightweight & Fast
- Multiple Load Balancing Methods
- Health Checks & Failover
- Keepalive / Connection Reuse

UPSTREAM BLOCKS (NGINX CONFIG)

```
upstream app_backend {
    server 10.0.0.11:8080;
    server 10.0.0.12:8080;
    server 10.0.0.13:8080;
}
```

CLIENTS ↔ NGINX (REVERSE PROXY) → 10.0.0.11:8080, 10.0.0.12:8080, 10.0.0.13:8080

MULTIPLE BACKEND SERVERS

LOAD BALANCING METHODS

ROUND ROBIN

Requests are distributed sequentially to each server in order.

LEAST CONNECTIONS

5 2 7

Requests go to the server with the least active connections.

IP HASH

CLIENT IP 203.0.113.10

A client IP is mapped to a specific server using a hash function.

WEIGHTED LOAD BALANCING

5 3 1

Servers receive traffic based on their assigned weights.

BACKEND WEIGHTS

WEIGHTS

server1 weight=5
server2 weight=3
server3 weight=1

Higher weight servers handle more requests proportionally.

BACKUP SERVERS

PRIMARY SERVERS | BACKUP SERVER (STANDBY)

Backup servers are used only when all primary servers are unavailable.

CONNECTION REUSE (KEEPALIVE CONNECTIONS)

NGINX ⇄ BACKEND SERVER

Keepalive reuses existing TCP connections to reduce latency and improve performance.

BACKEND FAILURE HANDLING

FAILED

NGINX automatically stops sending traffic to failed servers and routes to healthy ones.

LOAD-BALANCING ARCHITECTURE

INTERNET → DNS → NGINX (LOAD BALANCER) → APPLICATION SERVERS → DATABASE → CACHE

NGINX SITS AT THE FRONT, DISTRIBUTING TRAFFIC, TERMINATING TLS, AND PROTECTING BACKEND SERVICEES.

CHOOSING AN ALGORITHM

ALGORITHM	BEST FOR	CONSIDERATIONS
Round Robin	General purpose, even workloads.	Simple and effective when servers are similar.
Least Connections	Long-lived connections, APIs.	Better for uneven connection durations.
IP Hash	Session persistence without cookies.	Not ideal if client IPs are behind NAT.
Weighted Load Balancing	Servers with different capacities.	Distribute more traffic to stronger servers.
Backup Servers	High availability.	Used only when all primary servers fail.

CHOOSE THE RIGHT METHOD BASED ON YOUR APPLICATION PATTERN, TRAFFIC CAPACITY, SESSION REQUIREMENTS, AND FAILURE TOLERANCE.

MONITOR, TEST, AND ADJUST YOUR STRATEGY AS YOUR TRAFFIC GROWS.



TROUBLESHOOTING 502 AND 504 INCIDENTS

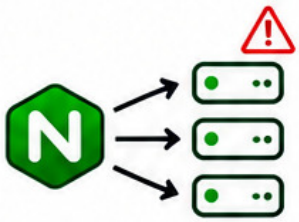
METHODOICAL INVESTIGATION = FASTER ROOT CAUSE + QUICKER RECOVERY

SCENARIO 1

NGINX RETURNS 502 BAD GATEWAY AFTER AN APPLICATION DEPLOYMENT

INVESTIGATE

- Backend process
- Upstream address
- Application port
- Unix sockets
- DNS
- Connection refusal
- Application crashes
- Logs



STEP-BY-STEP INVESTIGATION PATH

1. **CONFIRM THE ERROR**
```
curl -I https://yourdomain.com
# Expect: HTTP/1.1 502 Bad Gateway
```
2. **CHECK NGINX ERROR LOG**
```
$ sudo tail -f /var/log/nginx/error.log
```
3. **CHECK UPSTREAM STATUS**
```
$ sudo nginx -T | grep -A 20 upstream
```
4. **VERIFY BACKEND PROCESS IS RUNNING**
```
ps aux | grep your_app
systemctl status your_app
```
5. **CHECK BACKEND PORT LISTENING**
```
ss -tulpn | grep :<port>
netstat -tulpn | grep :<port>
```
6. **TEST CONNECTIVITY TO BACKEND**
```
curl -v http://127.0.0.1:<port>/health
curl -v http://<backend-ip>:<port>/health
```
7. **IF USING UNIX SOCKET**
```
ls -l /path/to/socket.sock
curl --unix-socket /path/to/socket.sock
http://localhost/health
```
8. **CHECK DNS RESOLUTION (IF USING HOSTNAMES)**
```
dig <backend-hostname>
nslookup <backend-hostname>
```
9. **LOOK FOR APPLICATION CRASHES**
```
journalctl -u your_app --since "10 min ago"
dmesg | tail -20
```
10. **RELOAD NGINX AFTER FIX (NOT RESTART)**
```
sudo nginx -t && sudo systemctl reload nginx
```

SCENARIO 2

USERS RECEIVE 504 GATEWAY TIMEOUT ONLY DURING PEAK TRAFFIC

INVESTIGATE

- Application latency
- Database latency
- Upstream timeouts
- Worker saturation
- Connection pools
- Backend capacity
- Dependency failures



STEP-BY-STEP INVESTIGATION PATH

1. **CONFIRM THE ERROR UNDER LOAD**
```
curl -I https://yourdomain.com
# Expect: HTTP/1.1 504 Gateway Timeout
```
2. **CHECK ERROR LOG FOR TIMEOUTS**
```
sudo tail -f /var/log/nginx/error.log
# Look for "upstream timed out"
```
3. **CHECK UPSTREAM RESPONSE TIME**
```
grep 'upstream_response_time' /var/log/nginx/access.log |
awk '{print $NF}' | sort -nr | head
```
4. **MONITOR BACKEND LATENCY**
```
curl -w "\nTotal Time: %{time_total}\n"
http://backend-ip:<port>/health
```
5. **CHECK DATABASE LATENCY**
```
mysqladmin -u user -p status
# or use DB monitoring tools
```
6. **CHECK NGINX WORKER SATURATION**
```
ps aux | grep nginx
cat /proc/$(pgrep -n nginx)/status | grep "Voluntary"|
```
7. **CHECK ACTIVE CONNECTIONS**
```
ss -s
ss -tanp | grep <backend-port> | wc -l
```
8. **CHECK UPSTREAM TIMEOUT CONFIGURATION**
```
nginx -T | grep -E "proxy_connect_timeout" \
proxy_read_timeout|proxy_send_timeout" -n
```
9. **CHECK BACKEND CAPACITY**
```
top -b -n 1 | head -20
free -h
df -h
```
10. **CHECK DEPENDENCIES (CACHE, QUEUE, API, ETC.)**
```
curl -I https://dependency-service/health
redis-cli ping   # Example for Redis
```

QUICK REFERENCE COMMANDS

NGINX TEST & RELOAD
- nginx -t
- nginx -T
- systemctl reload nginx
- systemctl status nginx

LOGS
- tail -f /var/log/nginx/access.log
- tail -f /var/log/nginx/error.log
- journalctl -u nginx -f

NETWORK & CONNECTIONS
- ss -tulpn
- ss -tanp
- netstat -tulpn
- dig domain.com

UPSTREAM TEST
- curl -v http://127.0.0.1:port
- curl -v http://<ip>:<port>
- curl --unix-socket /path/socket
- http://localhost/health

NGINX AS A REVERSE PROXY

ARCHITECTURE • CONFIGURATION • PRODUCTION BEST PRACTICES



REVERSE PROXY
Sits between clients and backend servers, and forwards requests securely.

LOAD DISTRIBUTION
Distributes incoming traffic across multiple application servers.

SECURITY & CONTROL
Hides backend servers, enforces access rules, and adds an extra security layer.

PERFORMANCE BOOST
Handles caching, compression, and optimized content delivery.

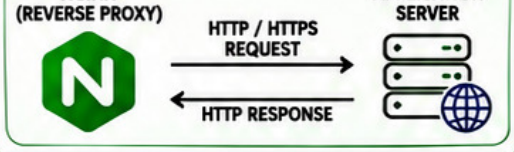
FLEXIBLE & POWERFUL
Supports proxying, SSL/TLS termination, header control, timeouts, and more.

REVERSE PROXY ARCHITECTURE



NGINX ACCEPTS CLIENT REQUESTS AND FORWARDS THEM TO BACKEND APPLICATION SERVERS. THE RESPONSE FROM THE SERVERS IS RETURNED TO THE CLIENT.

PROXYING TO APPLICATION SERVERS



PROXY_PASS

```
location / {
    proxy_pass  http://app_backend;
}
upstream app_backend {
    server  10.0.0.10:8080;
    server  10.0.0.11:8080;
    server  10.0.0.12:8080;
}
```

REQUEST HEADERS

HEADER	PURPOSE
Host	Preserves original host requested by client.
X-Real-IP	Sends the actual client IP to backend.
X-Forwarded-For	Sends the client IP chain through proxies.
X-Forwarded-Proto	Sends the original protocol (http or https).

EXAMPLE: PASSING CLIENT INFORMATION

```
location /  {
    proxy_pass   http://app_backend;

    proxy_set_header  Host  $host;
    proxy_set_header  X-Real-IP  $remote_addr;
    proxy_set_header  X-Forwarded-For  $proxy_add_x_forwarded_for;
    proxy_set_header  X-Forwarded-Proto  $scheme;
}
```

PROXY HTTP VERSIONS

SETTING	DESCRIPTION
proxy_http_version 1.1;	Required for keepalive, WebSockets, and HTTP/1.1 features.
proxy_set_header Connection "";	Clears the Connection header for proper keepalive.
proxy_set_header Upgrade $http_upgrade;	Enables protocol upgrades (WebSocket support).
proxy_set_header Connection $connection_upgrade;	Used with WebSocket proxying.

REQUEST BUFFERING

DIRECTIVE	DESCRIPTION
proxy_request_buffering on;	Buffers the client request body before sending to backend.
proxy_request_buffering off;	Streams the request body to backend without buffering.

RESPONSE BUFFERING

DIRECTIVE	DESCRIPTION
proxy_buffering on;	Buffers backend responses before sending to client.
proxy_buffering off;	Streams backend responses directly to client.

DISABLE BUFFERING FOR LARGE UPLOADS OR STREAMING TO REDUCE LATENCY AND MEMORY USAGE.

PROXY_PASS DEFINES WHERE NGINX SENDS THE REQUEST. IT CAN POINT TO A SINGLE SERVER OR AN UPSTREAM GROUP.

PROXY TIMEOUTS

DIRECTIVE	DESCRIPTION
proxy_connect_timeout 5s;	Time allowed to connect to upstream.
proxy_send_timeout 60s;	Time allowed to send request to upstream.
proxy_read_timeout 60s;	Time to wait for response from upstream.
keepalive_timeout 60s;	Time allowed to send response to client.

LOW TIMEOUTS CAN CAUSE 502 BAD GATEWAY AND GATEWAY TIMEOUT ERRORS.

TROUBLESHOOTING UPSTREAM COMMUNICATION

- CHECK BACKEND SERVER STATUS AND AVAILABILITY.
- VERIFY NETWORK CONNECTIVITY BETWEEN NGINX AND BACKEND.
- CHECK NGINX ERROR LOG FOR UPSTREAM ERRORS.
- CONFIRM CORRECT proxy_pass AND UPSTREAM CONFIGURATION.
- CHECK TIMEOUT SETTINGS.
- VERIFY DNS RESOLUTION IF USING DOMAIN NAMES.
- USE curl OR telnet TO TEST BACKEND DIRECTLY.
- MONITOR UPSTREAM RESPONSE TIMES AND FAILURES.

GOOD LOGS + CORRECT TIMEOUTS + HEALTHY BACKENDS = RELIABLE REVERSE PROXY SETUP.

ZERO-DOWNTIME DEPLOYMENTS AND TRAFFIC CONTROL

KEEP SERVICES AVAILABLE • CONTROL TRAFFIC • DEPLOY SAFELY



1 GRACEFUL CONFIGURATION RELOAD

- ✓ Reload without dropping connections
- ✓ Existing connections continue
- ✓ New connections use new configuration

```
sudo nginx -t &&
sudo systemctl reload nginx
```

2 TESTING BEFORE RELOAD

- ✓ Always test configuration
- ✓ Catch syntax errors
- ✓ Validate changes before applying

```
sudo nginx -t
```

3 REMOVING BACKENDS SAFELY

- ✓ Mark backend down before removing
- ✓ Stop new connections
- ✓ Let existing connections finish

```
server backup.example.com
max_fails=0;
down;
```

4 ROLLING DEPLOYMENTS

- ✓ Update instances one at a time
- ✓ Keep others serving traffic
- ✓ Zero downtime

Update → Drain → Remove → Deploy → Restore

5 BLUE-GREEN DEPLOYMENTS

BLUE (OLD) ⇄ GREEN (NEW)

- ✓ Two identical environments
- ✓ Switch traffic instantly
- ✓ Quick rollback by switching back

Canary → Monitor → Increase → Full Rollout

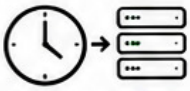
7 WEIGHTED TRAFFIC DISTRIBUTION



- ✓ Control traffic share between versions
- ✓ Useful for canary and gradual rollouts

weight=80 (old)
weight=20 (new)

8 CONNECTION DRAINING CONCEPTS



- ✓ Stop accepting new connections
- ✓ Allow existing connections to complete
- ✓ Prevent dropped requests

Use 'down' or remove backend after drain timeout

9 DEPLOYMENT VALIDATION



- ✓ Check application health
- ✓ Validate logs and metrics
- ✓ Confirm user experience
- ✓ Automated and manual checks

Health endpoints, synthetic checks, logs, metrics

10 FAILED RELEASE HANDLING



- ✓ Detect issues quickly
- ✓ Stop rollout immediately
- ✓ Protect users
- ✓ Communicate and mitigate

Alert → Stop → Rollback → Investigate

11 ROLLBACK



- ✓ Revert to last known good version
- ✓ Fast and reliable
- ✓ Part of every plan

Switch back / Restore previous config

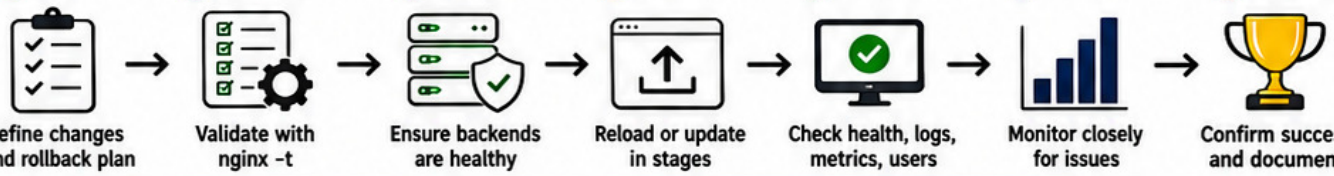
12 AVOIDING DROPPED CONNECTIONS



- ✓ Use graceful reloads
- ✓ Drain connections
- ✓ Keep timeouts appropriate
- ✓ Monitor during deploys

reload → drain → deploy → validate

PRODUCTION DEPLOYMENT WORKFLOW



Cloud Messenger™ | ZERO-DOWNTIME IS A PROCESS, NOT A SINGLE COMMAND. TEST, DEPLOY GRADUALLY, MONITOR, AND ROLLBACK FAST IF NEEDED.

NGINX WITH DOCKER AND KUBERNETES



MODERN DEPLOYMENT • SCALABLE • CLOUD NATIVE • PRODUCTION READY

1 NGINX IN CONTAINERS

- ✓ Lightweight and portable
- ✓ Consistent across environments
- ✓ Easy scaling and updates
  Ideal for microservices architectures

Docker Run Example

```
docker run -d --name nginx \
  -p 80:80 \
  -v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro \
  -v $(pwd)/html:/usr/share/nginx/html:ro \
  nginx:alpine
```

2 CONTAINER CONFIGURATION

- ✓ nginx.conf is the main file
- ✓ Custom includes in conf.d/
- ✓ Time worker processes and connections
- ✓ Optimize timeouts, buffers, and keepalive

Minimal nginx.conf Example

```
worker_processes auto;
events {
    worker_connections 1024;
}
http {
    include /etc/nginx/conf.d/*.conf;
}
```

3 VOLUMES

- ✓ Persist configuration, logs, TLS certificates, and content
- ✓ Use bind mounts or named volumes
- ✓ Keep containers stateless

Volume Example

```
-v $(pwd)/logs:/var/log/nginx
-v $(pwd)/certs:/etc/nginx/certs
-v $(pwd)/conf:/etc/nginx/conf.d
```

4 CONFIGURATION INJECTION

- ✓ Inject configs using volumes
- ✓ Use environment variables with templates
- ✓ ConfigMaps (Kubernetes)
- ✓ Immutable container images

Env Variable Example

```
docker run -e NGINX_HOST=example.com \
  -e NGINX_PORT=80 \
  nginx:alpine
```

5 REVERSE PROXY BETWEEN CONTAINERS

CLIENT → NGINX → APP 1 CONTAINER, APP 2 CONTAINER, APP 3 CONTAINER

- ✓ NGINX routes traffic to backend containers
- ✓ Service discovery via Docker DNS
- ✓ Load balance multiple application containers

Upstream Example

```
upstream app {
   server app1:3000;
   server app2:3000;
   server app3:3000;
}
```

6 DOCKER NETWORKING

01010 01010 — USER-DEFINED BRIDGE NETWORK

- ✓ Create user-defined networks
- ✓ Containers resolve by name
- ✓ Isolated and secure communication
- ✓ Expose only what is necessary

Create Network & Run

```
docker network create frontend
docker run --name nginx --network frontend -p 80:80 nginx
docker run --name app1 --network frontend app:latest
```

NGINX IN KUBERNETES

7 KUBERNETES INGRESS CONCEPTS

CLIENT → INGRESS → SERVICE, SERVICE, SERVICE

- ✓ HTTP/HTTPS routing layer
- ✓ Single entry point to cluster
- ✓ Rule-based traffic routing
- ✓ Works at Layer 7 (HTTP)

8 NGINX INGRESS CONTROLLER

NGINX INGRESS CONTROLLER

- ✓ Popular and production-ready
- ✓ High performance
- ✓ Advanced annotations
- ✓ Active community support

9 SERVICES AND UPSTREAM WORKLOADS

INGRESS → SERVICE, SERVICE, SERVICE

- ✓ Ingress routes to Services
- ✓ Services load balance Pods
- ✓ Backend endpoints change dynamically

10 TLS AT INGRESS

- ✓ Terminate TLS at Ingress
- ✓ Use Kubernetes Secrets
- ✓ Supports Lets Encrypt
- ✓ End-to-end encryption supported

11 ROUTING RULES

/api	→ SERVICE A
/web	→ SERVICE B
/example.com	→ SERVICE C

- ✓ Path-based routing
- ✓ Host-based routing
- ✓ Regex and rewrite rules
- ✓ Traffic splitting (canary)

12 LOGS AND TROUBLESHOOTING

- ✓ Check Ingress Controller logs
- ✓ kubectl logs -n ingress-nginx
- ✓ Use labeled describe ingress
- ✓ Verify Endpoints and Service
- ✓ Check events and annotations

13 NGINX REVERSE PROXY VS KUBERNETES INGRESS

NGINX REVERSE PROXY (STANDALONE)	KUBERNETES INGRESS (WITH NGINX INGRESS CONTROLLER)
Runs outside Kubernetes	Runs inside Kubernetes
Manually managed and configured	Managed as Kubernetes resource
Static upstream servers or DNS	Dynamic upstream via Services
Best for traditional infrastructure	Best for cloud-native workloads
Full control over configuration	Easy integration with K8S features
Scaling requires manual setup	Auto-scaling and self-healing
Not aware of Kubernetes objects	Understands Pods, Services, Endpoints

14 USEFUL COMMANDS

DOCKER

```
docker ps
docker logs <nginx_container>
docker run -d --name nginx_container -p 80:80 nginx
docker network ls
docker network inspect <network>
```

KUBERNETES

```
kubectl get pods -A
kubectl describe ingress <name> -n <ns>
kubectl get svc -A
kubectl apply -f ingress.yaml
kubectl get nodes
kubectl get pods -n ingress-nginx
```

Cloud Messenger™

Learn | Build | Deploy | Grow

Thank you for reading!

Created by: cloud-messenger™

WEBSITE	cloudmessenger.in
LINKEDIN	linkedin.com/in/cloud-messenger
GITHUB	github.com/Cloud-Messenger

#buildwithCloud-Messenger

Learn • Build • Visualize • Troubleshoot • Grow

Made with care by cloud-messenger™

COPYRIGHT NOTICE

© 2026 cloudmessenger. The compilation, structure, layout, design and branding of this workbook are the original work of cloudmessenger.

All rights reserved. Cloud Messenger™ is a trademark of cloud-messenger.